



RESEARCH ARTICLE

OPEN ACCESS

A Secure Hybrid Encryption-Based Framework for Controlled and Auditable File Sharing

Hardik Pawar^{*1}, Harsh Nimbare², Aryan Kasaker³, Nandini Baisane⁴ and Pooja Gharat⁵

^{1, 2, 3, 4, 5} St. John College of Engineering and Management - SJCEM. Palghar-Maharashtra, India.

¹ <http://orcid.org/xxxx-xxxx-xxxx-xxxx> , ² <http://orcid.org/xxxx-xxxx-xxxx-xxxx> , ³ <http://orcid.org/xxxx-xxxx-xxxx-xxxx> ,

⁴ <http://orcid.org/xxxx-xxxx-xxxx-xxxx> , ⁵ <http://orcid.org/xxxx-xxxx-xxxx-xxxx> 

Email: *hardikpawar000@gmail.com, nimbareharsh6@gmail.com, kasekararyan15@gmail.com, baisanenandini@gmail.com, poojapa@sjcem.edu.in

ARTICLE INFO

Article History

Received: Month 01th, 20XX

Accepted: Month 15th, 20XX

Published: Month 30th, 20XX

Keywords:

Cloud Security, Hybrid Encryption, Secure File Sharing, Access Control, Audit Logging

ABSTRACT

Cloud-based file storage systems offer scalability and flexibility, but they introduce significant challenges related to data security, privacy, and controlled access. This paper presents a secure file sharing framework designed for cloud-oriented environments by integrating hybrid encryption, user-level access control, and audit logging. The proposed system employs symmetric encryption using Fernet for efficient data protection, while asymmetric encryption using RSA is used for secure key management. A key feature of the framework is a secure key redistribution process that enables controlled sharing of encrypted data without directly exposing encryption keys to multiple users. The system also incorporates access control mechanisms and activity logging to enhance accountability and traceability of user actions. Experimental evaluation was conducted in a controlled environment to analyze encryption performance across different file sizes. The results indicate near-linear scalability with an average throughput of approximately 50–60 MB/s under the evaluated conditions. Although encryption introduces computational overhead, the system maintains a practical balance between security and performance. The proposed framework provides a structured and implementable approach for secure file storage and sharing in cloud environments.



Copyright ©2025 by authors and Galileo Institute of Technology and Education of the Amazon (ITEGAM). This work is licensed under the Creative Commons Attribution International License (CC BY 4.0).

I. INTRODUCTION

Cloud computing has become a fundamental paradigm for data storage and resource sharing, enabling scalable, flexible, and cost-efficient access to distributed computing resources. It is widely adopted in enterprise applications, collaborative platforms, and cloud-based services due to its ability to provide on-demand infrastructure and seamless data accessibility. However, the outsourcing of data to third-party cloud environments introduces significant concerns related to security, privacy, and loss of user control over sensitive information [1]-[3].

One of the primary challenges in cloud storage systems is ensuring the confidentiality and integrity of data stored on external servers. Since users do not have direct control over cloud infrastructure, stored data remains vulnerable to threats such as unauthorized access, data breaches, insider attacks, and information leakage. Prior research highlights that cloud environments are susceptible to such risks in the absence of robust protection mechanisms, emphasizing the need for secure storage and controlled data sharing solutions [1], [2].

To mitigate these challenges, encryption techniques are widely employed to secure data before outsourcing it to the cloud, ensuring confidentiality even in untrusted environments [5], [6]. In particular, hybrid encryption approaches that combine symmetric and asymmetric cryptographic techniques have gained attention for providing a balance between computational efficiency and security [10],

[13]. Despite these advancements, many existing solutions focus primarily on data encryption while overlooking critical aspects such as secure key distribution and fine-grained user-level access control. In several systems, encryption keys are directly shared or managed in ways that increase the risk of unauthorized access. While techniques such as proxy re-encryption provide mechanisms for key delegation [9], they often introduce additional complexity and are not always integrated into practical file-sharing systems.

In addition to data protection, ensuring controlled and secure access to shared data remains a major challenge. Inefficient key management and inadequate access control mechanisms can lead to unauthorized data exposure, even when encryption is applied. Another limitation observed in many existing approaches is the lack of activity monitoring and audit logging, which restricts the ability to track user actions and detect potential security violations. These limitations highlight the need for an integrated framework that combines encryption, key management, access control, and system-level monitoring within a unified architecture.

To address these challenges, this paper proposes a secure file sharing framework designed for cloud-oriented environments. The proposed system integrates hybrid encryption with user-level access control and audit logging to provide a comprehensive security solution. Symmetric encryption is used for efficient data protection, while asymmetric cryptography is employed for secure key management. The framework incorporates a secure key redistribution process, where encryption keys are protected separately for each authorized user, eliminating the need for direct key sharing. This approach improves control over data access and reduces the risk of key exposure.

The system also supports controlled file sharing and maintains detailed audit logs to record user activities such as file uploads, downloads, sharing, and deletion operations. By integrating these components, the proposed framework provides a practical and secure solution for cloud-based file sharing. While similar techniques have been explored in prior work, this study focuses on integrating these mechanisms into a functional implementation that addresses both security and usability concerns. The main contributions of this work are summarized as follows:

1. Design and implementation of a secure file sharing system integrating hybrid encryption with user-level access control and activity monitoring.
2. Development of a secure key redistribution approach for controlled key management without direct key sharing.
3. An end-to-end framework enabling secure file upload, controlled sharing, and protected data access in cloud-oriented environments.

II. THEORETICAL REFERENCE

Cloud data security and privacy have been widely studied due to the increasing reliance on cloud-based storage systems. Early research focused on identifying the risks associated with outsourcing data to third-party environments. Studies highlight issues such as data breaches, unauthorized access, insider threats, and lack of user control, emphasizing the need for robust protection mechanisms in cloud storage systems [1]–[3]. These challenges demonstrate that data confidentiality and integrity cannot be guaranteed without the use of dedicated security frameworks.

Various approaches have been proposed to enable secure data storage and sharing in cloud environments. Existing frameworks rely on encryption-based protection to ensure confidentiality of outsourced data and support controlled sharing among users [4]–[6]. These approaches demonstrate the importance of encrypting data before storage and maintaining protection in untrusted environments. However, many solutions focus primarily on data encryption while providing limited support for efficient key management and practical sharing mechanisms.

Access control and secure data sharing mechanisms have also been extensively explored. Fine-grained access control models enable flexible and secure data sharing among users, particularly in dynamic cloud environments [7]. Lightweight secure data sharing schemes have been proposed to improve efficiency in mobile cloud scenarios [8]. Techniques such as proxy re-encryption provide mechanisms for secure delegation of access rights without exposing underlying data [9]. While these approaches improve key management, they often introduce additional computational complexity and may not be directly suitable for practical implementations.

Hybrid encryption techniques have gained attention for cloud storage applications due to their ability to balance security and efficiency. Several studies demonstrate the effectiveness of combining symmetric and asymmetric encryption for secure cloud-based file storage [10]–[12]. More recent work enhances these approaches by incorporating adaptive key management and secure access mechanisms to improve both performance and security [13]. These methods highlight the importance of combining efficient data encryption with secure key protection strategies.

Despite these advancements, achieving an effective balance between security, efficiency, and practical implementation remains a challenge. Existing studies indicate that many solutions lack integrated handling of encryption, key management, and access control within a unified framework [14], [15]. In addition, limited attention is given to activity monitoring and audit logging, which are essential for ensuring accountability and traceability in real-world systems. These limitations motivate the development of the proposed framework, which focuses on integrating hybrid encryption, controlled key management, access control, and logging mechanisms into a single system. This approach aims to provide a practical and secure solution for cloud-based file storage and sharing.

III. MATERIALS AND METHODS

III.1 SYSTEM OVERVIEW

The proposed secure file sharing framework is designed to provide a reliable and efficient mechanism for storing and sharing files in cloud-oriented environments. The system integrates hybrid encryption, user authentication, access control, and activity monitoring to ensure the confidentiality, integrity, and traceability of stored data.

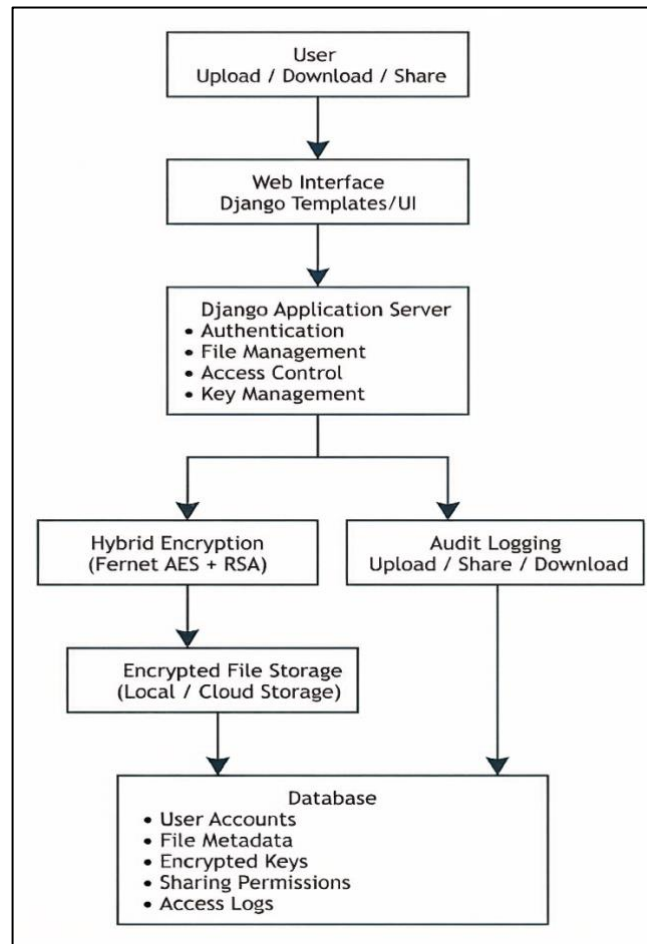


Figure 1: System Architecture of the Proposed Framework
Source: Authors, (2026).

The system consists of several key components, including the user interface, application server, encryption module, database, and file storage system. Users interact with the system through a web-based interface that enables secure file upload, sharing, and retrieval operations, while the application server handles authentication and coordinates communication between system components.

In the proposed framework, when a user uploads a file, it is processed through the encryption module before being stored. A hybrid encryption approach is used, where symmetric encryption using Fernet (AES-based) protects the file content, while asymmetric encryption using RSA secures the corresponding encryption keys. This approach allows efficient data encryption while maintaining secure key management.

The encrypted files are stored in the file storage system, while metadata such as file ownership, encrypted keys, access permissions, and timestamps are maintained in the database. Encryption keys are stored only in encrypted form, which reduces the risk of key exposure.

The framework also supports controlled file sharing between users. During sharing, encryption keys are redistributed using a user-specific key re-encryption process, so that only authorized users can access the shared data. Access control mechanisms further restrict file access based on user permissions.

The system includes an activity logging module that records all major operations, including file uploads, downloads, sharing, and deletion events, along with timestamps and user-related information. This improves transparency and allows tracking of user actions.

Overall, the proposed architecture combines secure storage, controlled data sharing, and activity monitoring within a unified framework, making it suitable for practical deployment in cloud-based environments.

III.2 SECURE FILE UPLOAD

The secure file upload process is designed to ensure that all data is protected before being stored in the system. In the proposed framework, the upload operation begins with user authentication, where only registered and authorized users are allowed to access the upload functionality. This step prevents unauthorized entities from introducing or accessing sensitive data within the system.

Once a file is selected for upload, it is processed through the encryption module prior to storage. The system follows a hybrid encryption approach, combining symmetric and asymmetric cryptographic techniques to achieve both efficiency and security. A unique symmetric key is generated dynamically for each file using a secure random generation process. This key is then used to encrypt the file content using the Fernet scheme, which is based on the Advanced Encryption Standard (AES) and provides authenticated encryption.

The use of symmetric encryption allows efficient processing of large files, as it requires significantly less computational overhead compared to asymmetric methods. After the file content is encrypted, the generated symmetric key is secured using asymmetric encryption. In this step, the symmetric key is encrypted using the public key of the file owner, ensuring that only the corresponding private key holder can access it. This design prevents unauthorized exposure of encryption keys while maintaining secure key storage.

After encryption, the encrypted file is stored in the file storage system, while associated metadata such as file ownership, encrypted keys, file identifiers, and timestamps are maintained in the database. This separation of encrypted data and metadata improves system organization and supports efficient access control mechanisms.

The upload process also ensures that encryption keys are never stored in plaintext form within the system. Keys are handled only during processing and are stored in encrypted form, reducing the risk of key compromise. Even if the storage environment is exposed, the encrypted files remain protected due to the absence of accessible plaintext data or keys.

The secure file upload mechanism establishes the foundation for protected data storage by ensuring confidentiality at the time of data entry. It also prepares the system for controlled file sharing, where encryption keys can later be securely redistributed to authorized users without exposing the original key.

III.3 SECURE FILE SHARING

The secure file sharing mechanism enables controlled distribution of encrypted files among authorized users while maintaining data confidentiality and enforcing strict access restrictions. In the proposed framework, file sharing is initiated by the data owner, who specifies the intended recipient(s) through the application interface. The system ensures that only registered and authenticated users can participate in the sharing process.

A key aspect of the framework is the secure redistribution of encryption keys without directly exposing them. Instead of sharing the original symmetric encryption key, the system follows a user-specific key re-encryption process. This approach ensures that each authorized user receives a uniquely protected version of the encryption key, preventing direct key sharing between users.

When a file is shared, the system retrieves the encrypted symmetric key associated with the file from the database. This key is initially encrypted using the public key of the file owner during the upload phase. To enable sharing, the system temporarily decrypts the symmetric key using the owner's private key within a controlled environment. The recovered symmetric key is then re-encrypted using the public key of the intended recipient. As a result, the recipient obtains a version of the encryption key that can only be decrypted using their corresponding private key.

The re-encrypted key is stored alongside the sharing metadata in the database, ensuring that each sharing instance maintains a separate and secure key mapping. This design binds access rights to individual users and prevents unauthorized users from accessing encryption keys, even if they have access to stored data.

The system also enforces access validation before allowing any file retrieval. When a user attempts to access a shared file, the system verifies whether the user has the necessary permissions. Only after successful validation does the system provide access to the encrypted file and the corresponding encrypted key. The recipient then decrypts the symmetric key using their private key and subsequently decrypts the file content.

All file sharing activities are recorded in the logging module to maintain accountability and traceability. The logs capture details such as the file owner, recipient, type of operation, timestamps, and associated user information. This enables monitoring of system usage and supports detection of unauthorized or suspicious activities.

The secure file sharing mechanism ensures that data remains protected throughout the distribution process. By combining controlled access, secure key redistribution, and activity logging, the framework provides a practical and secure approach for sharing encrypted data in cloud-oriented environments.

III.4 AUDIT LOGGING

The proposed system incorporates an audit logging mechanism to enhance transparency, accountability, and security monitoring within the file sharing framework. The logging module is designed to record all significant user activities, allowing the system to maintain a detailed and structured history of operations performed on stored data.

Each interaction with the system is captured as a log entry. This includes operations such as file uploads, downloads, sharing events, and file deletion actions. For every recorded event, the system stores relevant information including the user identity, file identifier, type of operation performed, timestamp, and associated network information such as the user's IP address. This structured approach to logging enables precise tracking of user behavior and system usage over time.

The inclusion of detailed logging plays an important role in strengthening system security. In case of unauthorized access attempts or abnormal activity patterns, the stored logs can be analyzed to reconstruct the sequence of events. This allows administrators to identify potential security breaches and take appropriate corrective actions. Since each action is linked to a specific authenticated user, the system enforces accountability and discourages misuse of resources.

The log data is stored securely within the system database, ensuring that it is protected from unauthorized access or modification. Access to audit logs can be restricted to authorized administrative users, which prevents exposure of sensitive operational information. The logging mechanism is integrated with the overall system architecture and operates alongside encryption and access control components to provide an additional layer of security.

The audit logging feature enhances the practical usability of the framework by providing visibility into system operations. It also supports monitoring, debugging, and post-incident analysis, making it an essential component for secure and controlled file management in cloud-oriented environments.

III.5 CRYPTOGRAPHIC DESIGN

The proposed framework adopts a hybrid cryptographic design to ensure secure data storage and controlled access in cloud-oriented environments. The design combines symmetric and asymmetric encryption techniques to achieve a balance between computational efficiency and strong security, which is essential for handling large files while maintaining confidentiality.

For data encryption, the system utilizes symmetric cryptography due to its high performance in processing large volumes of data. A unique symmetric key is generated dynamically for each file using a secure random generation method. This key is used to encrypt the file content using the Fernet scheme, which is based on the Advanced Encryption Standard (AES) and provides authenticated encryption. The use of Fernet ensures both confidentiality and integrity of the stored data by combining encryption with message authentication.

To securely manage encryption keys, asymmetric cryptography is incorporated into the system. During the upload phase, the symmetric key associated with each file is encrypted using the public key of the file owner. This ensures that only the owner, possessing the corresponding private key, can access the original symmetric key. By separating data encryption from key protection, the system reduces the risk of key exposure and strengthens overall security.

During the file sharing process, the system applies a controlled key redistribution approach. The encrypted symmetric key is first decrypted using the owner's private key in a secure execution environment. The recovered key is then re-encrypted using the public key of the authorized recipient. This process ensures that each user receives a distinct encrypted version of the symmetric key, which can only be accessed using their private key. As a result, direct sharing of encryption keys is avoided, and access rights are tightly bound to individual users.

The asymmetric encryption operations are performed using the RSA algorithm with secure padding mechanisms, which enhances resistance against cryptographic attacks. Since asymmetric encryption is applied only to the symmetric key rather than the entire file, the computational overhead remains low while maintaining strong security guarantees.

The hybrid cryptographic design enables efficient handling of large data volumes while ensuring secure key management and controlled access. By combining symmetric encryption for data protection and asymmetric encryption for key security, the framework provides a practical and robust solution for secure file storage and sharing in cloud-based systems.

III.6 HYBRID ENCRYPTION WORKFLOW

The hybrid encryption workflow of the proposed system illustrates the complete process of secure file handling, including file upload, key protection, secure sharing, and file retrieval. The overall workflow is depicted in Fig. 2.

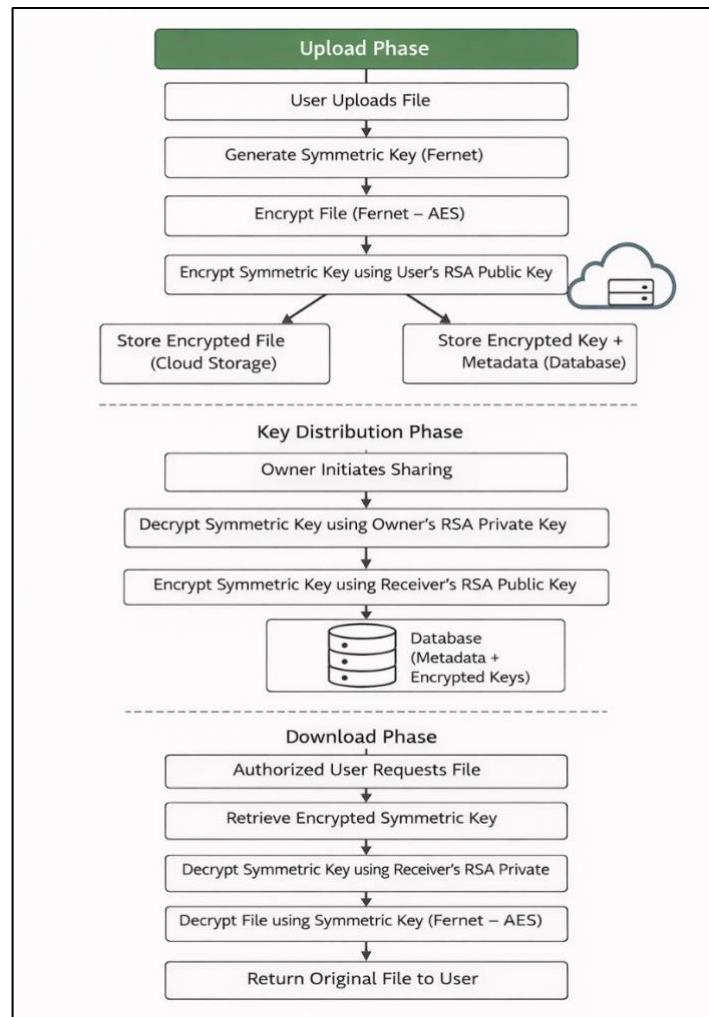


Figure 2: Hybrid Encryption Workflow of the Proposed Framework
Source: Authors, (2026).

The workflow begins with the file upload phase. When a user selects a file for storage, a unique symmetric key is generated using a secure random generation process. This key is used to encrypt the file content using the Fernet scheme, ensuring confidentiality and integrity of the data. After encrypting the file, the symmetric key is secured by encrypting it with the public key of the file owner. This ensures that only the owner can access the original encryption key.

The next stage involves the file sharing process. When the file owner decides to share a file with another user, the system retrieves the encrypted symmetric key associated with the file. The key is temporarily decrypted using the owner’s private key within a controlled execution environment. The recovered symmetric key is then encrypted again using the public key of the intended recipient. This results in a user-specific encrypted key, which ensures that only the designated recipient can access it using their private key.

During the file retrieval phase, an authorized user requests access to a shared file. The system first validates the user’s permissions to ensure that the request is legitimate. After successful validation, the encrypted file and the corresponding encrypted symmetric key are provided to the user. The user decrypts the symmetric key using their private key and then uses the decrypted key to recover the original file content.

All operations performed within the workflow are recorded by the audit logging module. This includes file uploads, sharing actions, downloads, and deletion events, along with associated user and network information. The logging mechanism ensures traceability of actions and supports monitoring of system usage.

The hybrid workflow integrates encryption, secure key management, controlled access, and activity logging into a unified process. This structured approach ensures that data remains protected throughout its lifecycle, from initial upload to final access, while maintaining efficiency and usability in cloud-oriented environments.

III.7 IMPLEMENTATION DETAILS

The proposed secure file sharing framework is implemented using a web-based architecture to ensure accessibility, scalability, and ease of deployment. The system is developed using the Python programming language with the Django framework, which provides a structured backend for handling user authentication, file management, and communication between system components. The web-based interface allows users to interact with the system through standard browsers, enabling file upload, sharing, and retrieval operations.

The cryptographic functionality of the system is implemented using the Python *cryptography* library. Symmetric encryption is performed using the Fernet module, which is based on the Advanced Encryption Standard (AES) and provides authenticated encryption for file data. This ensures both confidentiality and integrity of stored files. A unique encryption key is generated for each file using a secure random generation mechanism. Asymmetric encryption is implemented using the RSA algorithm for secure key management. RSA encryption is applied to protect the symmetric keys, ensuring that only authorized users with the corresponding private keys can access them.

The system architecture maintains a clear separation between file storage and metadata management. Encrypted files are stored in a dedicated storage location, while associated metadata such as file identifiers, encrypted keys, ownership details, access permissions, and timestamps are maintained within the database. This separation reduces the risk of data exposure and supports efficient access control and retrieval operations.

The application also incorporates a logging module that records all significant user activities. These include file uploads, downloads, sharing actions, and deletion events. Each log entry contains details such as user identity, operation type, timestamps, and network-related information, enabling detailed monitoring and analysis of system usage. The logs are securely stored within the database and can be accessed for auditing and administrative purposes.

The implementation supports flexibility in deployment by allowing the use of different database systems such as SQLite for development and PostgreSQL for production environments. File storage can be configured for local storage or integrated with cloud-based storage services, depending on deployment requirements. Table 1 summarizes the key technologies and components used in the implementation of the proposed system.

Table 1: Technology Stack Used in the Proposed Framework

Component	Technology Used
Backend	Django (Python)
Encryption	RSA and Fernet (AES)
Database	SQLite / PostgreSQL
Storage	Local / Cloud Storage
Logging	Custom Logging Module

Source: Authors, (2026).

IV. RESULTS AND DISCUSSIONS

IV.1 EXPERIMENTAL SETUP

The performance evaluation of the proposed secure file sharing framework was conducted in a controlled local environment to analyze its efficiency and computational overhead. The system was implemented and tested on a machine equipped with an Intel Core i5 processor, 8 GB RAM, and running a Windows operating system. The application was deployed using the Django development server, and all experiments were executed under consistent system conditions.

The evaluation was performed in a local setup to isolate computational performance from network-related factors such as latency and bandwidth variations. This allowed accurate measurement of the encryption overhead introduced by the proposed system without interference from external conditions.

To evaluate the performance of the hybrid encryption mechanism, files of varying sizes were used as input. The selected file sizes ranged from small to large files, covering typical use cases in file storage and sharing systems. This variation in input size enabled analysis of how the system performs under different data loads.

The primary metric considered in the evaluation was encryption time. For each file, the total time required to complete the encryption process was recorded. This includes the time taken for symmetric encryption of the file content using Fernet as well as the asymmetric encryption of the corresponding symmetric key using RSA. Measuring both components together provides a realistic estimate of the total processing overhead introduced by the hybrid encryption approach.

Each experiment was conducted multiple times to ensure consistency in the results. The average values were considered to minimize the impact of system-level fluctuations and background processes. The collected data was then used to analyze the relationship between file size and encryption time, which serves as a key indicator of the scalability and efficiency of the proposed framework.

The experimental setup was designed to reflect practical operating conditions while maintaining controlled parameters for accurate performance evaluation.

IV.2 PERFORMANCE EVALUATION

The encryption performance of the proposed framework was evaluated by measuring the time required to encrypt files of varying sizes. The results obtained from the experimental setup are presented in Table 2.

Table 2: Encryption Time vs File Size

File Size (MB)	Encryption Time (sec)
1	0.0235
100	1.64
600	11.97
1148	22.59

Source: Authors, (2026).

The results show that the encryption time increases with the size of the input file. This trend is expected, as the symmetric encryption process operates on the entire file content, leading to higher processing time for larger files. The contribution of asymmetric encryption remains relatively small since it is applied only to the encryption of the symmetric key.

The observed performance indicates a near-linear relationship between file size and encryption time. This suggests that the system scales in a predictable manner when handling increasing data volumes. Based on the recorded values, the system achieves an approximate throughput in the range of 50–60 MB/s, which remains consistent across different file sizes.

The results also highlight the trade-off between security and performance. While encryption introduces additional computational overhead compared to non-encrypted operations, this overhead remains within acceptable limits for typical file sizes. The use of symmetric encryption for data processing ensures efficiency, while asymmetric encryption is limited to key protection, reducing its overall impact on execution time.

These observations indicate that the proposed framework provides a practical balance between performance and security. The system is capable of handling typical file storage and sharing operations without significant degradation in processing time under the evaluated conditions.

IV.3 DISCUSSION

The experimental results indicate that the proposed hybrid encryption-based framework provides a practical balance between security and performance for cloud-oriented file sharing systems. The observed near-linear increase in encryption time with respect to file size suggests that the system scales in a predictable manner when handling larger data volumes. This behavior is consistent with the characteristics of symmetric encryption, which accounts for the majority of the computational cost during processing.

The measured throughput of approximately 50–60 MB/s shows that the system maintains stable performance across different file sizes under the evaluated conditions. This suggests that the inclusion of asymmetric encryption for key protection does not introduce significant overhead, as it is applied only to the encryption of the symmetric key rather than the entire file content.

From a security perspective, the framework strengthens data protection by combining hybrid encryption with user-level access control and audit logging. The key redistribution process ensures that encryption keys are not directly shared between users. Instead, each authorized user receives a separately protected version of the symmetric key, which reduces the risk of unauthorized key exposure. This approach improves control over key access compared to systems where keys are reused or shared directly.

When compared with existing approaches discussed in the literature, many systems address encryption or access control as separate components. The proposed framework focuses on integrating encryption, secure key handling, access control, and activity logging within a unified architecture. This integration improves both system usability and operational security, making it more suitable for practical deployment scenarios.

The evaluation also highlights certain limitations. The experiments were conducted in a controlled local environment, which does not fully represent performance in distributed or real-world cloud deployments where network latency and system load can affect execution time. The system introduces computational overhead due to encryption operations, which may become more noticeable in resource-constrained environments or when processing very large datasets.

The results suggest that the proposed framework achieves a reasonable trade-off between security and performance while addressing key challenges in secure file sharing. Future improvements can focus on optimizing performance for large-scale deployments and enhancing key management strategies to further strengthen the system.

V. CONCLUSIONS

This paper presented a secure file sharing framework designed for cloud-oriented environments by integrating hybrid encryption, user-level access control, and audit logging to address key security challenges in data storage and sharing systems. The proposed approach combines symmetric encryption using Fernet for efficient data protection with asymmetric encryption using RSA for secure key management. The framework incorporates a secure key redistribution process that enables controlled sharing of encrypted data without directly exposing encryption keys to multiple users. In addition to encryption, the system includes access control and logging mechanisms to support controlled data sharing and improved visibility of user activities. The experimental results indicate that the proposed system exhibits near-linear scalability in encryption performance while maintaining stable throughput under the evaluated conditions. Although encryption introduces computational overhead, the system maintains a reasonable balance between security and efficiency. The evaluation was conducted in a controlled local environment, which does not fully reflect performance in large-scale distributed cloud systems. Future work can focus on optimizing system performance for real-world deployments and enhancing key management strategies to further strengthen the framework.

VI. AUTHOR'S CONTRIBUTION

Conceptualization: Hardik Pawar, Harsh Nimbare and Aryan Kasekar.

System Design: Hardik Pawar and Harsh Nimbare.

Implementation: Hardik Pawar, Aryan Kasekar and Nandini Baisane.

Experimental Evaluation: Harsh Nimbare and Aryan Kasekar.

Analysis and Interpretation: Hardik Pawar and Harsh Nimbare.

Writing – Original Draft: Hardik Pawar.

Writing – Review and Editing: Harsh Nimbare and Pooja Gharat.

Supervision: Pooja Gharat.

Approval of the final text: All authors.

VIII. REFERENCES

- [1] P. Yang, N. Xiong and J. Ren, "Data Security and Privacy Protection for Cloud Storage: A Survey," *IEEE Access*, vol. 8, pp. 131723–131740, 2020, doi: 10.1109/ACCESS.2020.3009876.
- [2] I. Gupta, A. K. Singh, C.-N. Lee and R. Buyya, "Secure Data Storage and Sharing Techniques for Data Protection in Cloud Environments: A Systematic Review, Analysis, and Future Directions," *IEEE Access*, vol. 10, pp. 71247–71277, 2022, doi: 10.1109/ACCESS.2022.3188110.
- [3] U. A. Butt, R. Amin, M. Mehmood et al., "Cloud Security Threats and Solutions: A Survey," *Wireless Personal Communications*, vol. 128, pp. 387–413, 2023, doi: 10.1007/s11277-022-09960-z.
- [4] B. S. Rawal and S. S. Vivek, "Secure Cloud Storage and File Sharing," 2017 IEEE International Conference on Smart Cloud (SmartCloud), New York, NY, USA, 2017, pp. 78–83, doi: 10.1109/SmartCloud.2017.19.
- [5] O. A. Khashan, "Secure Outsourcing and Sharing of Cloud Data Using a User-Side Encrypted File System," *IEEE Access*, vol. 8, pp. 210855–210867, 2020, doi: 10.1109/ACCESS.2020.3039163.
- [6] M. Ali et al., "SeDaSC: Secure Data Sharing in Clouds," *IEEE Systems Journal*, vol. 11, no. 2, pp. 395–404, Jun. 2017, doi: 10.1109/JSYST.2014.2379646.
- [7] S. Xu, G. Yang, Y. Mu and R. H. Deng, "Secure Fine-Grained Access Control and Data Sharing for Dynamic Groups in the Cloud," *IEEE Transactions on Information Forensics and Security*, vol. 13, no. 8, pp. 2101–2113, Aug. 2018, doi: 10.1109/TIFS.2018.2810065.
- [8] R. Li, C. Shen, H. He, X. Gu, Z. Xu and C.-Z. Xu, "A Lightweight Secure Data Sharing Scheme for Mobile Cloud Computing," *IEEE Transactions on Cloud Computing*, vol. 6, no. 2, pp. 344–357, Apr.–Jun. 2018, doi: 10.1109/TCC.2017.2649685.
- [9] Z. Qin, H. Xiong, S. Wu and J. Batamuliza, "A Survey of Proxy Re-Encryption for Secure Data Sharing in Cloud Computing," *IEEE Transactions on Services Computing*, doi: 10.1109/TSC.2016.2551238.
- [10] K. Negi, R. Shrestha, T. L. Borges, S. Sahana and S. Das, "A Hybrid Cryptographic Approach for Secure Cloud-Based File Storage," 2023 IEEE IAS Global Conference on Emerging Technologies (GlobConET), London, United Kingdom, 2023, pp. 1–7, doi: 10.1109/GlobConET56651.2023.10150148.
- [11] G. Viswanath and P. V. Krishna, "Hybrid Encryption Framework for Securing Big Data Storage in Multi-Cloud Environment," *Evolutionary Intelligence*, vol. 14, pp. 691–698, 2021, doi: 10.1007/s12065-020-00404-w.
- [12] V. Verma, P. Kumar, R. K. Verma and S. Priya, "A Novel Approach for Security in Cloud Data Storage Using AES-DES-RSA Hybrid Cryptography," 2021 Emerging Trends in Industry 4.0 (ETI 4.0), Raigarh, India, 2021, pp. 1–6, doi: 10.1109/ETI4.051663.2021.9619274.
- [13] D. Shivaramakrishna and M. Nagaratna, "A novel hybrid cryptographic framework for secure data storage in cloud computing: Integrating AES-OTP and RSA with adaptive key management and time-limited access control," *Alexandria Engineering Journal*, vol. 84, pp. 275–284, 2023.
- [14] B. Seth, S. Dalal, V. Jaglan, D.-N. Le, S. Mohan and G. Srivastava, "Integrating encryption techniques for secure data storage in the cloud," *Transactions on Emerging Telecommunications Technologies*, vol. 33, no. 1, p. e4108, 2022, doi: 10.1002/ett.4108.
- [15] K. Sasikumar and S. Nagarajan, "Comprehensive Review and Analysis of Cryptography Techniques in Cloud Computing," *IEEE Access*, vol. 12, pp. 52325–52351, 2024, doi: 10.1109/ACCESS.2024.3385449.